数据库原理

Database Principle

陶宏才，李晓航

西南交通大学

计算机与人工智能学院

第9章 关系数据库设计理论

学习目的和要求

- 关系模式中可能存在的问题
- 为何存在这些问题
- 函数依赖
- 关系模式的规范形式
- 关系模式的规范化



9.1 关系模式中可能存在的问题

一.存在问题的关系模式示例

(1) 示例模式1

Students (Sid, SName, DName, DDirector, Cid, CName, CScore)

该关系模式中包含的数据语义：

① 系与学生之间是1：n的联系；

② 系与系主任之间是1：1的联系；

③ 学生与课程之间是m：n的联系。

假定：系及系主任信息再无单独的关系模式分别存放。

Students (Sid, SName, DName, DDirector, Cid, CName, CScore)

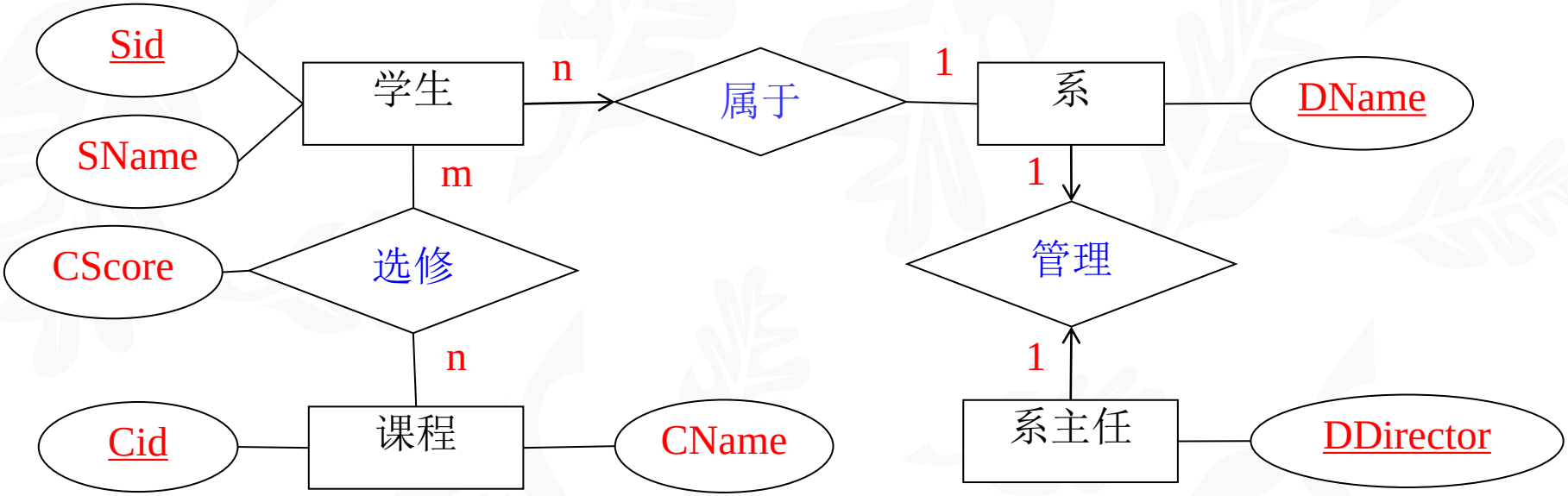


图6-1 由Students模式逆向工程转换的ERM

(2) 示例模式2

STC (Sid, Tid, Cid)

该关系模式中包含的数据语义：

① 课程与教师之间为1：n的联系；

② 学生与课程之间为m：n的联系。

候选键为：(Sid, Cid)和(Sid, Tid)。

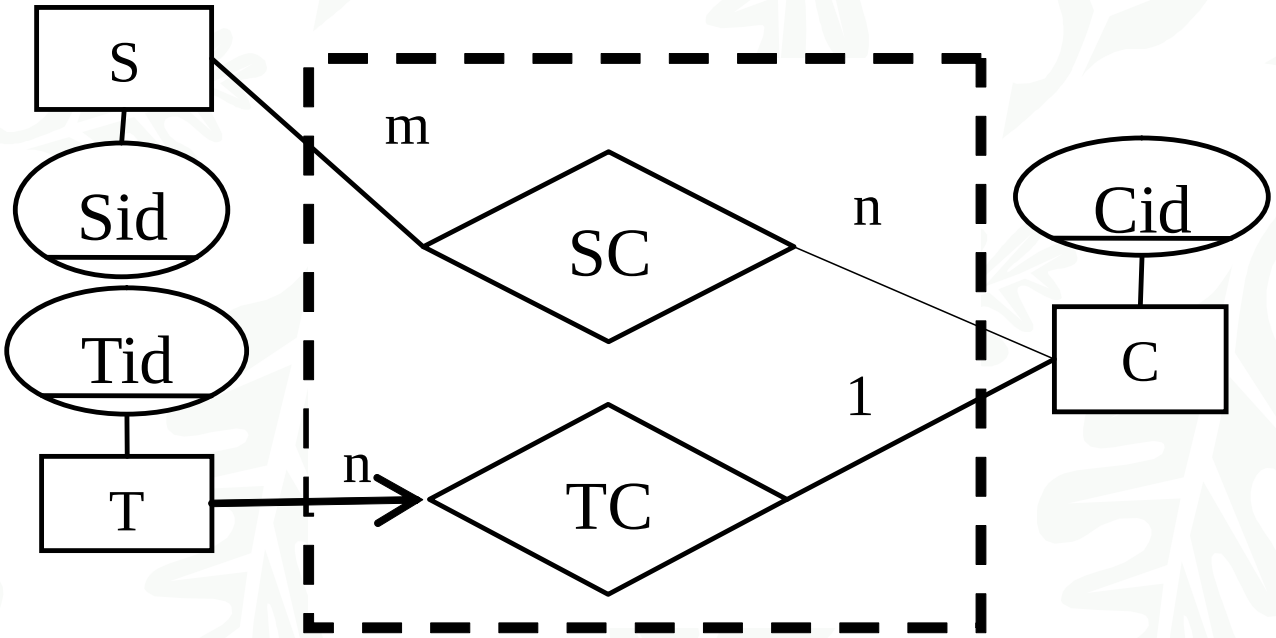


图 6-6 STC 关系由虚线框内两个联系整体转换得到

二.存在的问题

(1) 插入异常(Insert Anomaly)

表现: ① 元组插不进去;

② 插入一个元组却要求插入多个元组。

(2) 删除异常(Delete Anomaly)

表现: ① 删除时删掉了其他信息;

② 删除一个元组却删除了多个元组。

(3) 冗余(Redundancy)

表现: ① 某种信息在关系中存储多次;

(4) 更新异常(Update Anomaly)

表现: ① 更新一个元组却要求更新多个元组。

9.2 为何存在这些问题

数据的语义不但在完整性方面有体现，在关系模式的设计方面也有体现。

具体表现： 在关系模式中的属性间存在一定的依赖关系，此即数据依赖。

数据依赖（Data Dependency）： 指通过一个关系中属性间值的相等与否体现出来的数据间的相互关系。

数据依赖分类： 函数依赖(Functional Dependency, FD)、多值依赖(Multivalued Dependency，MVD)和连接依赖(Join Dependency，JD)。

数据依赖决定因素： 由现实系统中属性间相互联系的语义决定。

异常现象产生的根源： 关系模式中属性间存在的这些依赖关系。

根源的体现及解决： 如果将各种数据集中于一个关系模式中，各对象主键云集，极可能无法全部满足各主键约束等，从而造成异常。

解决异常方法： 利用规范化理论，对关系模式进行相应的分解，以消除这些异常。

9.3 函数依赖

一.函数依赖定义

约定：设R是一关系模式，U是R的属性集合，X、Y⊆U，r是R的一个关系实例，元组 t∈r。则用t[X]表示元组t在属性集合X上的值。同时，将关系模式和关系实例统称为关系，XY表示X和Y的并集。（一个例子见下页）

函数式决定：$y=f(x)$，说明$x \rightarrow y$

函数依赖定义：设R是一个关系模式，U是R的属性集合，X和Y是U的子集。对于R的任意实例r，r中任意两个元组t_1和t_2，如果$t_1[X]=t_2[X]$ 则$t_1[Y]=t_2[Y]$，那么称X函数地决定Y，或Y函数地依赖于X，记作：$X \rightarrow Y$，X称为决定子(Determinant)/决定属性集。

函数依赖关心的问题：是一个或一组属性的值决定其他属性的值。

例：上一页各种符号（R、U、X、Y、r、t）的例子

R：R (Sid, SName, DName, DDirector)

U：{ Sid, SName, DName, Ddirector }　　X、Y⊆U

r：{<001, 姓名1, 部门1, 导师1>, <002, 姓名2, 部门1, 导师1>, <003, 姓名3, 部门1, 导师1>, <004, 姓名4, 部门2, 导师2>, <005, 姓名5, 部门2, 导师2> , <006, 姓名6, 部门2, 导师2> }

假设 t = <002, 姓名2, 部门1, 导师1>,

X ={Sid, Sname}, Y= { Sid, DName } , Z = { Sid }

则：t[X] = <002, 姓名2>, t[Y] = <002, 部门1>, t[Z]=<002>

二.函数依赖分类及其定义

函数依赖类型：

① 平凡函数依赖(Trivial FD)

② 非平凡函数依赖(Nontrivial FD)

③ 完全函数依赖(Full FD)

④ 部分函数依赖(Partial FD)

⑤ 传递地函数依赖(Transitive FD)

平凡函数依赖：如果$Y \subseteq X$，则$X \rightarrow Y$称为平凡函数依赖。平凡函数依赖不反映新的语义。

非平凡函数依赖：如果$X \rightarrow Y$，且Y不是X的子集，则称$X \rightarrow Y$是非平凡函数依赖。如不特别声明，一般总是讨论非平凡函数依赖。

决定属性集/决定子：如果$X \rightarrow Y$，则称X为该函数依赖的决定属性集。

XY等价：如$X \rightarrow Y$，且$Y \rightarrow X$，则X与Y一一对应，记作$X \leftrightarrow Y$。

完全函数依赖： 设R是一个具有属性集合U的关系模式，如果X→Y，并且对于X的任何一个真子集Z，Z→Y都不成立，则称Y完全函数依赖于X，记作：$X \xrightarrow{f} Y$。

X=(a, b), Y=(c)，如果$a \not\rightarrow c$，$b \not\rightarrow c$，但(a, b)→c，则$X \xrightarrow{f} Y$。

部分函数依赖： 若X→Y，但Y不完全函数依赖于X，则称Y部分函数依赖于X，记作：$X \xrightarrow{p} Y$。

X=(a, b), Y=(c)，如果a→c或b→c，则$X \xrightarrow{p} Y$。

问题： 如果X为单个属性，则X→Y是完全函数依赖还是部分函数依赖？

传递函数依赖： 设R是一个具有属性集合U的关系模式，X、Y、Z⊆U，X、Y、Z是不同的属性集。如果X→Y，Y→X不成立，Y→Z，则称Z传递地函数依赖于X。

X→Y，Y↛X，Y→Z，则X→Z。



问题： X→Y，Y↛X，Y→Z，但Z→Y，那么X→Z？

三.函数依赖示例

针对示例模式1

Students (Sid, SName, DName, DDirector, Cid, CName, CScore)

根据其语义所得的函数依赖有：

❶ Sid→SName

❷ Sid→DName

❸ Cid→CName

❹ DName→DDirector

❺ DDirector→DName

❻ (Sid, Cid) $\xrightarrow{f}$ CScore

❼ (Sid, Cid) $\xrightarrow{p}$ SName

❽ (Sid, Cid) $\xrightarrow{p}$ DName

❾ (Sid, Cid) $\xrightarrow{p}$ CName

注意：这些函数依赖的存在，完全是由数据的语义决定的。

四.Armstrong公理系统

问题提出： 在关系模式的规范化处理过程中，不仅要知道一个给定的函数依赖集合，还要知道由给定的函数依赖集合所蕴涵（或推导出）的所有函数依赖的集合。为此，需要一个有效而完备的公理系统，Armstrong公理系统即这样的一个系统。

蕴含定义： 设F是R上的函数依赖集合，X→Y是R的一个函数依赖 。如果R的一个关系实例满足F，则必然满足X→Y，则称F逻辑蕴含(Imply) X→Y。（例：假设F = { A→B, B→C }，则F蕴含A→B, B→C 和A→C ）

闭包定义： 函数依赖集合F所逻辑蕴含的函数依赖的全体，称为F的闭包(Closure)，记为F^+。

Armstrong公理： 为从已知的函数依赖推导出其他的函数依赖，Armstrong提出了一套推理规则，称为Armstrong公理(Armstrong's Axioms)。

公理包含如下三条推理规则：

(1) 自反律(Reflexivity)：若Y⊆X⊆U，则X→Y。

(2) 增广律(Augmentation)：若X→Y，Z⊆U，则XZ→YZ。

(3) 传递律(Transitivity)：若X→Y和Y→Z，则X→Z。

引理 1：Armstrong公理是正确的，即：如F成立，则由F根据Armstrong公理所推导的函数依赖总是成立的。

引理 2：如下三条推理规则是正确的：

(1) 合并规则(Union)：如果X→Y，X→Z，则X→YZ。

(2) 伪传递规则(Pseudo Transitivity)：如果X→Y，YW→Z，则XW→Z。

(3) 分解规则(Decomposition)：如果X→Y，Z⊆Y，则X→Z。

或：如X→YZ，则X→Y，X→Z。

附：属性闭包（Attribute Closure）

- 1. 定义
  - 给定关系模式 R (U, F)，其中 U 是属性集，F 是函数依赖集。
  - 对于属性子集 $X \subseteq U$，X 的闭包 X^+ 是指通过 F 中的函数依赖，从 X 出发能够唯一确定的最大属性集。
- 2. 计算步骤
  - 1）初始化：令 $X^+ = X$（初始闭包就是 X 本身）。
  - 2）迭代扩展：遍历所有函数依赖 $Y \rightarrow Z \in F$。若 $Y \subseteq X^+$，则将 Z 加入 X^+（即 $X^+ = X^+ \cup Z$）。
  - 3）重复迭代：不断重复步骤 2，直到 X^+ 不再变化。
  - 4）终止：最终的 X^+ 即为所求。

附：属性闭包（Attribute Closure）（续）

- 3. 核心应用
  - 判断函数依赖：若 $Y \subseteq X^+$，则函数依赖 $X \rightarrow Y$ 成立。
  - 求解候选键：若 $X^+ = U$（全集）且 X 是最小集，则 X 是候选键。
  - 判断范式：验证函数依赖是否满足 BCNF/3NF 条件（如检查决定因素是否为超键）。
- 4. 例子：关系模式：R(A,B,C,D)，函数依赖集：$F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow A\}$，计算 AB^+：
  - 第 1 轮：初始$AB^+ = \{A, B\}$。由于 $AB \rightarrow C$，$AB \subseteq AB^+$，加入 C ，因此 $AB^+ = \{A, B, C\}$
  - 第 2 轮：由于 $C \rightarrow D$，$C \subseteq AB^+$，加入 D，因此 $AB^+ = \{A, B, C, D\}$
  - 第 3 轮：由于 $D \rightarrow A$：$D \subseteq AB^+$，但 A 已在闭包中，无需更新。
  - 终止：$AB^+ = \{A, B, C, D\}$

9.4 关系模式的规范形式

一.范式

范式(Normal Form, NF)：关系模式的规范形式。

关系模式中的范式：1NF、2NF、3NF、BCNF、4NF和5NF。

范式之间存在的关系或级别：

$$1NF \supset 2NF \supset 3NF \supset BCNF \supset 4NF \supset 5NF$$

函数依赖范畴　　多值依赖范畴　　连接依赖范畴

范式级别与异常问题之关系：一般，级别越低，出现异常的程度越高。

二.规范化

规范化：将一个给定的关系模式转化为某种范式的过程称为关系模式的规范化过程，简称**规范化**(Normalization)。

规范化方法：一般采用**分解**的办法，将低级别范式向高级别范式转化，使关系的语义单纯化。

规范化目的：逐渐消除异常。

理想的规范化程度：范式级别越高则规范化程度也越高。

实际的规范化操作：

① 1NF和2NF一般作为规范化过程的过渡范式。

② 规范程度不一定越高就越好。

③ 设计中一般达到3NF或BCNF即可。

三.以函数依赖为基础的范式

以函数依赖为基础的范式种类：第一范式、第二范式、第三范式和BCNF范式。

1.第一范式（1NF）

定义：设R是一个关系模式。如果R的每个属性的值域都是不可分的简单数据项的集合，则称该关系模式为第一范式关系模式，记作1NF。

商用RDBMS要求：关系的属性是原子的，即要求关系起码应为第一范式。

非第一范式的关系转换为1NF关系：将复合属性变为简单属性即可。

INF关系可能的异常：满足1NF是不够的，仍可能出现插入、删除和更新异常。因为可能存在“部分函数依赖”与“传递函数依赖”。例如前面的关系模式示例都是INF，但仍存在异常。

1NF异常示例：

Students (Sid, SName, DName, DDirector, Cid, CName, CScore)

其中的函数依赖有：

❶ Sid→SName　　❷ Sid→DName

❸ Cid→CName

❹ DName→DDirector　　❺ DDirector→DName

❻ (Sid, Cid) $\xrightarrow{f}$ CScore　　❼ (Sid, Cid) $\xrightarrow{p}$ SName

❽ (Sid, Cid) $\xrightarrow{p}$ DName　　❾ (Sid, Cid) $\xrightarrow{p}$ CName

异常：

(1)插入异常：插入学生，但学生还未选课，则不能插入；因为主键为(sid, cid)。

(2)删除异常：如某学生只选了一门课，现要删除学生的该门选课，则该学生信息亦被删除。

(3)冗余：如某学生选修了多门课程，则存在有多行多个字段值的重复存储。

(4)更新异常：由于存在冗余，故如果某学生要转系，则要修改多行。

2. 第二范式（2NF）

定义：若关系模式R是1NF，而且每一个非键属性都完全函数依赖于R的键，则称该关系模式为第二范式关系模式，记作2NF。

2NF实质：不存在非键属性“部分函数依赖”于键的情况。

非2NF关系或1NF关系向2NF的转换：消除其中的部分函数依赖，一般是将一个关系模式分解成多个2NF的关系模式。即：将部分函数依赖于键的非键属性及其决定属性移出，另成一关系，使其满足2NF。

分解示例：

Students (Sid, SName, DName, DDirector, Cid, CName, CScore)

(1) Students (Sid, SName, DName, DDirector)

(2) ElectiveC (Sid, Cid, CScore)

(3) Course (Cid, CName)

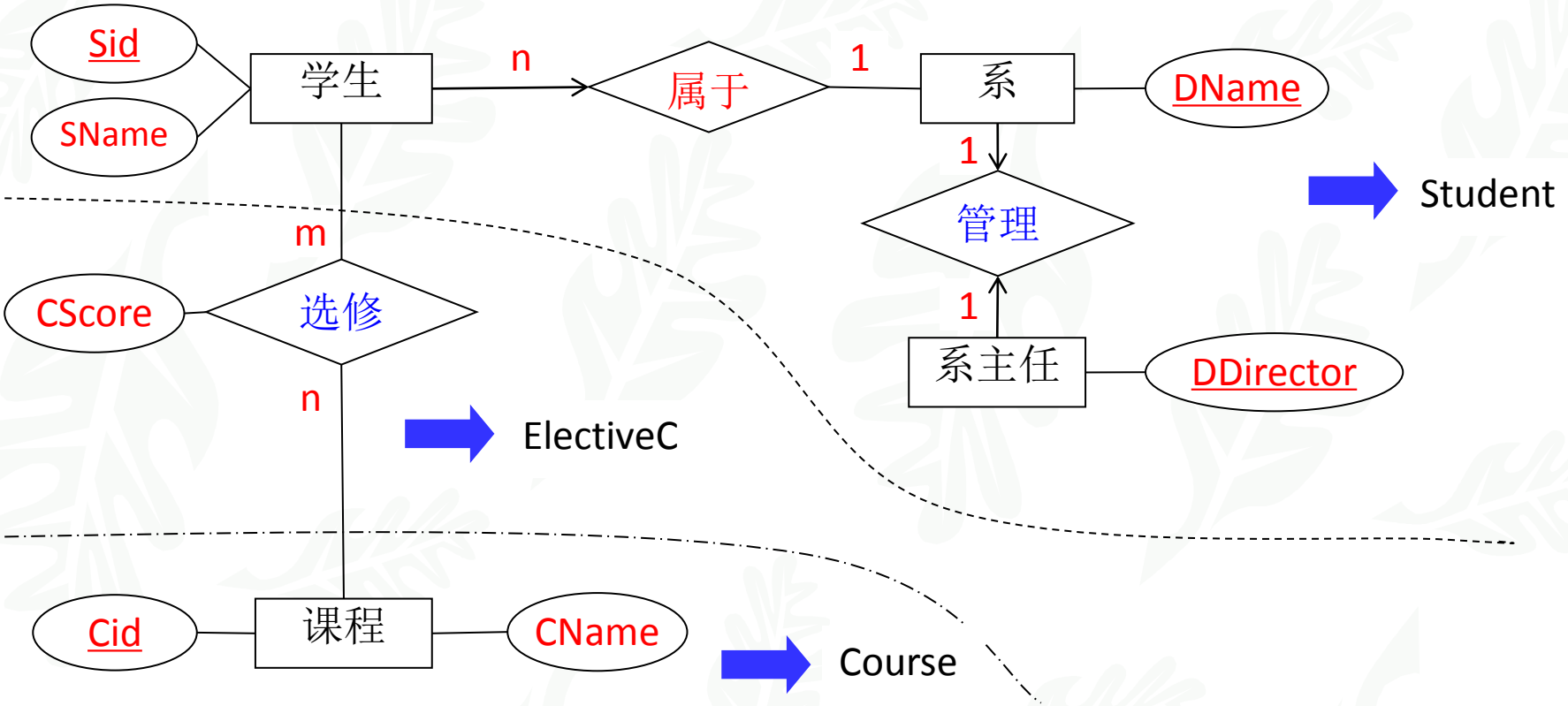


图6-2 将图6-1分为3个区域分别转换为相应的关系模式

2NF关系可能的异常：仍可能存在插入异常、删除异常、更新异常和冗余。因为，还可能存在“传递函数依赖”。

2NF异常示例：

Students (Sid, SName, DName, DDirector)

其中的函数依赖有：

❶ Sid→SName　　❷ Sid→DName

❸ DName→DDirector　　❹ Sid→DDirector

❺ DDirector→DName

异常：

(1)插入异常：插入尚未招生的系时，不能插入。

(2)删除异常：如某系学生全毕业，删除学生则会删除系的信息。

(3)冗余：由于系有众多学生，而每个学生均带有系信息，故冗余。

(4)更新异常：由于存在冗余，故如修改一个系信息，则要修改多行。

3. 第三范式（3NF）

定义：若关系模式R是2NF，而且它的任何一个“非键属性”都不传递依赖于R的任何候选键，则称该关系模式为第三范式关系模式，记作3NF。

3NF的得来： 3NF是从1NF消除非键属性对键的部分函数依赖和从2NF消除传递函数依赖而得到的关系模式。

2NF关系向3NF的转换：消除传递函数依赖，将2NF关系分解成多个3NF关系模式。

分解示例：

Students (Sid, SName, DName, DDirector)

(1) Students (Sid, SName, DName)

(2) Dept (DName, DDirector)

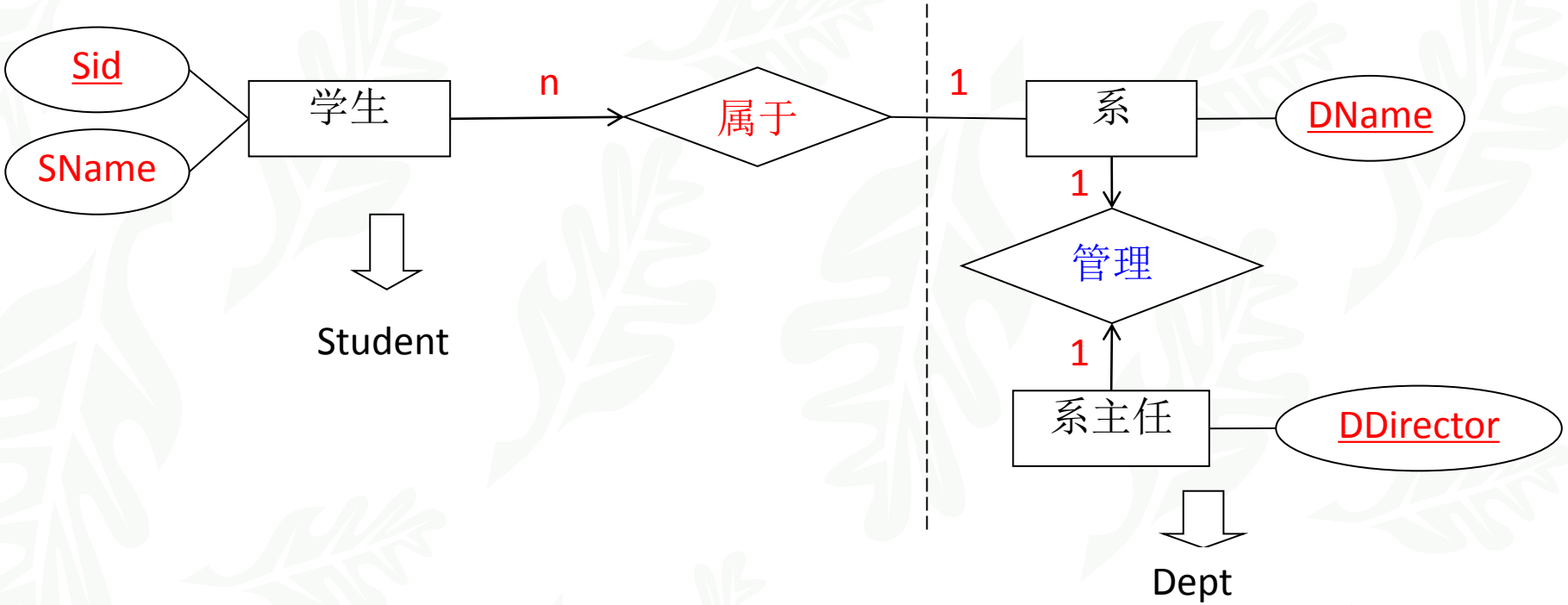


图6-4 将图6-2第一个区域又分为2个小区域分别转换为相应的关系

3NF关系可能的异常：仍可能存在插入异常、删除异常、更新异常和冗余。因为，还可能存在“主属性”“部分函数依赖”于键。

主属性：组成候选键的属性。

3NF异常示例：

STC (Sid, Tid, Cid)

其中的函数依赖有：

① (Sid, Cid) $\xrightarrow{f}$ Tid　　② (Sid, Tid) $\xrightarrow{P}$ Cid

③ Tid→Cid（注意：Cid是主属性，因此STC满足3NF）

异常：

(1)插入异常：插入尚未选课的学生时，不能插入。或插入没有学生选课的课程时，不能插入。

(2)删除异常：如选修某课程的学生全毕业，删除学生则会删除教师上该课程的信息。

(3)冗余：每个选修某课程的学生均带有教师id的信息，故冗余。

(4)更新异常：如修改某门课程的id号，则要修改多行。

4. BCNF范式

提出：由Boyce和Codd提出，故称BCNF范式。亦被认为是增强的第三范式，有时也归入第三范式。

定义：若关系模式R是1NF，如果对于R的每个函数依赖X→Y，X必为候选键，则R为BCNF范式（Boyce-Codd Normal Form, BCNF）。

每个BCNF范式具有的三个性质：

(1) 所有非键属性都完全函数依赖于每个候选键；

(2) 所有键属性都完全函数依赖于每个不包含它的候选键；

(3) 没有任何属性完全函数依赖于非键的任何一组属性。

说明：

(1) 由于BCNF的每一个非平凡函数依赖的决定子必为候选键，故不会出现3NF中决定子可能不是候选键的情况。

(2) 3NF不一定是BCNF，而BCNF一定是3NF。不过，属于3NF而非BCNF的关系模式不多，即使有，对数据库设计者来说，所引起的更新异常也不太重要。

(3) 3NF和BCNF常常都是数据库设计者所追求的关系范式。有些文献有时统称它们为第三范式，只要不引起误解。

(4) 如果一个关系数据库的所有关系模式都属于BCNF，那么，在函数依赖范畴内，它已达到了最高的规范化程度(但不是最完美的范式)，在一定程度上已消除了插入和删除的异常。

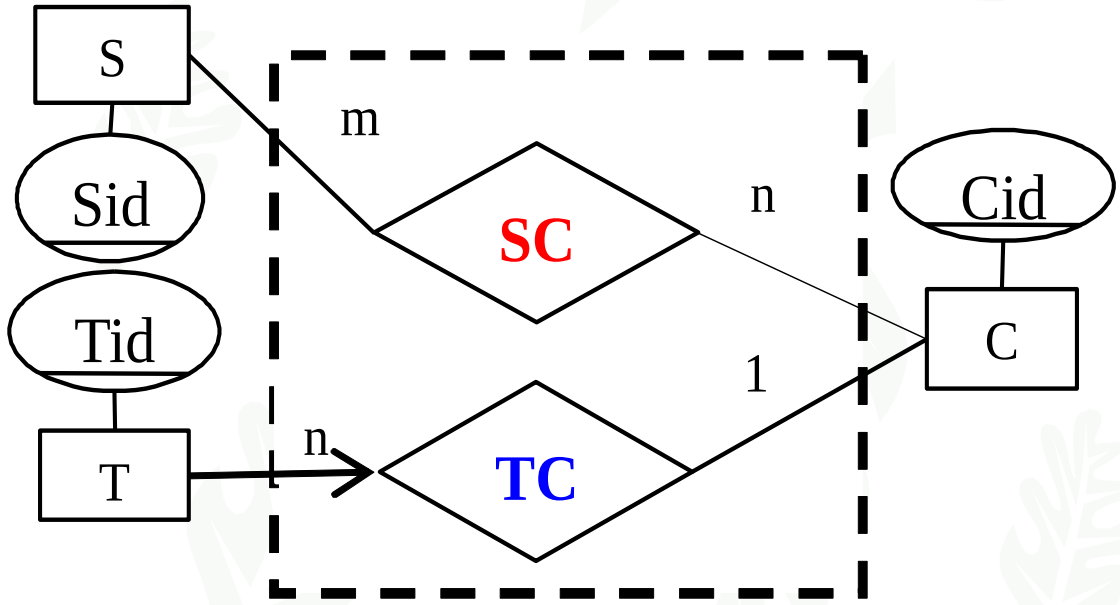
3NF关系向BCNF的转换：消除那些决定子不是超键的函数依赖，即可将3NF关系分解成多个BCNF关系模式。

分解示例：

STC (Sid, Tid, Cid)

(1) SC (Sid, Cid)

(2) TC (Tid, Cid)



9.5 关系模式的规范化

一.规范化步骤

规范化的实质： 概念的单一化。即：一个关系只描述一个概念、一个实体或实体间的一种联系，若多于一个概念就应将其它概念分离出去。

规范化基本步骤：



二.关系模式的分解及其指标

投影分解法： 一个关系模式R<U, F>，其中，U为该关系R的属性集，F为该关系R上的数据依赖，分解为若干个关系模式R_1<U_1, F_1>，R_2<U_2, F_2>...，R_n<U_n, F_n>，其中，$U=U_1 \cup U_2 \cup ... \cup U_n$，且$U_i \not\subset U_j$，$R_i$为R在$U_i$上的投影，此即意味着将存储于一张表T中的数据分散到若干张表T_1，T_2，...，T_n中去，其中，T_i是T在属性集U_i上的投影。

关系模式分解的一般要求： 关系模式经分解后，应与原来的关系等价。等价是指两者对数据的使用者来说是等价的，即：对分解前后的数据，做同样内容的查询，会产生同样的结果。

分解的两个指标： 无损分解和函数依赖保持性。分解不能丢失任何信息，否则就没有意义。

无损分解： 若R与R_1，R_2，...，R_n自然连接的结果相等，则称关系模式R的这个分解具有**无损连接性**（Loss-less Join）。只有具有无损连接性的分解才能够保证不丢失信息，无损连接性的分解即为**无损分解**。

保持函数依赖：若F所逻辑蕴含的函数依赖一定也由分解得到的某个关系模式中的函数依赖Fi所逻辑蕴含，则称关系模式R的这个分解是保持函数依赖（Preserve Dependency）的。

关系模式的分解准则：

① 只满足无损分解要求；

② 既满足无损分解要求，又满足保持依赖要求。



—— The End ——